D5 – Optimization and Ethics Report

Boréal Marché Customer Intelligence Platform | Manish Chauhan (ID# 6401170) | Individual | Companion files in the repository: GOVERNANCE_LOI25.md, MODEL_CARD.md, DATA_CARD.md; scripts/perf_profile.py, scale_probe.py, importance.py, fairness_audit.py

1. Summary

This report measures the deployed system (live on AWS ECS Fargate, repository tag v0.9) and what it costs, explains how the model reaches its scores, audits it for gaps between customer groups, and documents the Law 25 governance. Four results matter most:

- **Performance.** Single-prediction latency fell from 110 ms to 35 ms (3.2x) after one change: the forest was saved with `n_jobs=-1`, which starts a thread pool on every call; serving it single-threaded is faster at every batch size, with identical outputs. Loading the model at start-up (already done) saves about 65 ms per request versus loading per request.
- **Scalability.** One API task handles roughly 12–14 requests/s (estimate). The first things to break are the container-local SQLite prediction log (about 500 writes/s, and not shared between tasks) and the monitoring summary, which costs about 1 s of CPU per refresh once 5,000 rows are logged.
- **Fairness.** On the held-out slice every customer in spend quartiles Q1–Q3 and in the 0–90-day tenure band is predicted "not retained" (recall 0%), because the model never scores them above 0.52 while the cut-off is 0.82. Six of ten groups are flagged under the 5-point rule. Group thresholds close the spend gap at a precision cost; recalibration and reweighting did **not** help (reported as negative results).
- **Governance.** A human approves every retention offer, which is the design consequence of Law 25 s.12.1; the audit also recommends removing `country`, which adds no accuracy.

2. Performance optimisation

What was measured. `scripts/perf_profile.py` times, in process, model load, the first (cold) and warm prediction, batch scaling and explanation cost; with `--api URL` it times each live endpoint over the network (p50/p95/p99, sequential, one client). In-process numbers below come from a 2-vCPU development machine; the container has 0.5 vCPU, so absolute times there are larger and the ratios are what transfer. Live endpoint numbers are in Table 2.

Table 1. Latency profile, in process.

Measurement (dev machine, ms)	p50	p95	Note
Python imports (joblib, scikit-learn, app)	782		paid once per process
Model load at start-up	175		paid once; /ready turns green only after it
First prediction after load (cold)	41		vs 35 ms warm: little cold penalty after load
Load model on every request (rejected design)	100	148	includes the cold first call each time
Warm single prediction, n_jobs=-1 (as saved)	110.5	126.9	before
Warm single prediction, n_jobs=1 (as served now)	35.0	46.0	after: 3.2x faster, same output
Batch of 100: n_jobs=-1 / n_jobs=1	95.1 / 30.5	119.9 / 31.3	per item 0.95 ms → 0.31 ms
Batch of 1,000: n_jobs=-1 / n_jobs=1	131.1 / 45.9	142.8 / 61.3	per item 0.13 ms → 0.05 ms
Explanation: one call per factor group / one batched call	285.7 / 39.2	292.6 / 56.4	7.3x faster by batching

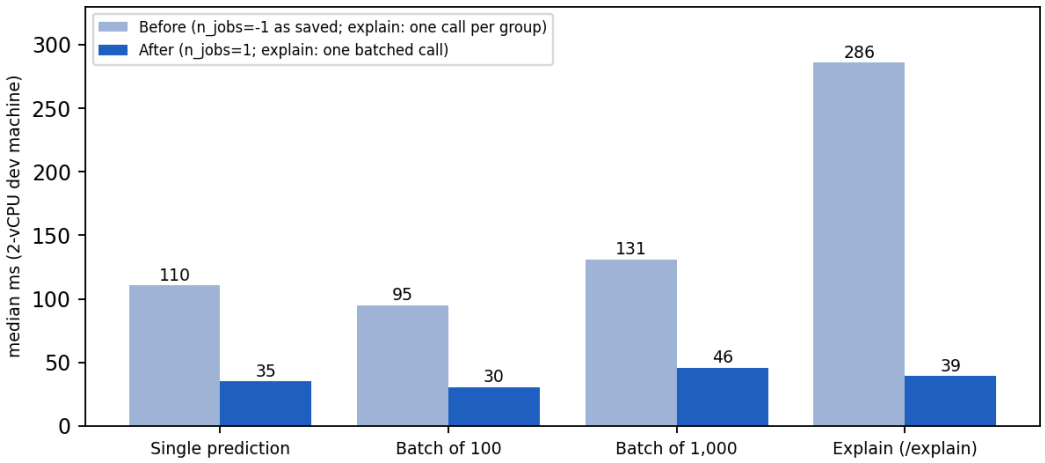


Figure 1. Before and after for the three changes (dev machine). Single/batch bars: forest as saved (n_jobs=-1) vs served single-threaded; explain bars: eight separate calls vs one batched call.

What changed and why. (1) **Start-up loading** was already in the design (the model is loaded once in the FastAPI lifespan, and a bad artifact fails the container before `/ready` goes green). Measured, per-request loading would add about 65 ms (100 ms vs 35 ms) and a ~175 ms load spike. (2) **Single-threaded forest.** The saved Extra Trees has 450 trees and `n_jobs=-1`; scikit-learn then spawns worker threads on every `predict_proba`, which costs more than it saves for one row and even for 1,000 rows. `ModelService` now sets `n_jobs=1` at load; concurrency comes from the web server. The golden-prediction regression test still passes, proving identical outputs. (3) **Explanations** score all factor groups in one batched call.

Endpoint (live, over HTTPS)	Before p50	Before p95	After p50	After p95
GET /health	52.3	72.1	63.0	81.0
GET /ready	58.0	68.8	63.6	81.9
GET /version	65.5	94.3	65.9	83.5
POST /predict	265.3	307.5	109.5	141.2
POST /explain	not deployed (404)	-	229.8	329.0
POST /predict/batch (100 customers)	972.9	1,795.1	826.4	934.0
POST /recommend	67.9	86.0	68.9	84.7
POST /chat (rule provider)	65.4	78.5	71.2	87.0

Table 2. Live API latency in milliseconds, 60 sequential requests per endpoint, measured on 26 Oct 2026 from my laptop over the internet. Before = the deployed v0.9 build; after = the build with the D5 changes (merge commit eddef8f). The round trip to the service (about 50-65 ms, see /health) is included in every figure.

Reading the live results. A prediction fell from a median of 265 ms to 109 ms (2.4x), and the 95th percentile from 308 ms to 141 ms. Subtracting the round trip shown by /health, server time went from roughly 210 ms to roughly 45 ms; that subtraction is an estimate. The change is the single-threaded forest: on a half-vCPU container the thread pool cost more than it saved. A batch of 100 customers improved less (p95 1.8 s to 0.9 s). /explain, which scores nine variants of a customer in one call, has a median of 230 ms and p95 of 329 ms, inside the 500 ms alert threshold. The endpoints that do not run the model (/health, /ready, /version, /recommend, /chat) did not change beyond normal variation, as expected. This is one client, one location and sequential requests, so it says nothing about throughput under concurrent load; that was not measured.

Cold start of the service. A deployment takes about 20 minutes end to end (build, push, ECS rollout; Deploy runs #1–#8 took 20m32s–20m59s). Once a container starts, imports plus model load take roughly 1 s on the dev machine, so the user-visible gap is the platform's, not the model's. With one task per service there is no zero-downtime guarantee during a rollout.

3. Scalability analysis

Estimates assume the deployed configuration: API task 0.5 vCPU / 1 GB, UI task 0.25 vCPU / 0.5 GB, one task each, a single uvicorn worker, SQLite log on the container disk.

Growth axis	Where it breaks	Estimate	What to change first
Traffic (requests/s)	Single worker, 0.5 vCPU. A prediction needs about 35 ms of CPU, plus JSON and one log write; the same work takes about 2x longer on half a core.	About 12–14 req/s per task (about 1M predictions/day in theory). Estimate, not load-tested on Fargate.	Add uvicorn workers or tasks behind the load balancer, but only after the log store is shared (next row).
Prediction log writes	SQLite, one file on one container: measured about 500 writes/s with a commit per row; not shared between tasks, wiped at every redeploy.	Breaks at about 500 predictions/s, and immediately when a second task is added (each would have its own monitoring state).	Move the log to managed PostgreSQL (the code path exists: DATABASE_URL=postgres...).
Monitoring summary (dashboard refresh)	Loads up to 5,000 rows and runs pandas PSI/KS for 28 features. Measured: 152 ms at 1,000 rows, about 1.0 s at 5,000 rows, and about 1.0 s at 20,000 rows because of the cap (LogStore.recent_predictions limit=5000).	One refresh = about 1 s of CPU, competing with predictions on the same worker. Beyond 5,000 predictions the 24 h window silently becomes 'the last 5,000'.	Compute drift in a scheduled job and cache the result; add per-hour sampling; surface the cap in the UI.
Customers (rows)	Training and retraining are in memory (5,249 customers, seconds). Batch scoring is linear: about 0.05 ms per item at 1,000 rows.	Fine to about 1M customers per scoring run; retraining memory becomes the limit near 10^6 rows with 450 trees.	Subsample training data or move retraining to a larger runner/SageMaker.
Association rules	Antecedent-indexed dictionary; measured 0.02 ms for a 50-item basket with 351 rules.	Lookups stay in microseconds up to about 10^5 rules; memory is the limit, not time.	Nothing until rule count passes about 10^5.
Assistant	Latency and cost are dominated by the LLM provider (network plus tokens), not the API.	Rule provider: sub-millisecond lookup. LLM providers: seconds per answer, rate-limited by the vendor.	Cache frequent questions; stream (already implemented).

Table 3. Break points. Measured values come from scripts/perf_profile.py and scripts/scale_probe.py; throughput on Fargate is an estimate until load-tested.

4. Cost analysis

Pricing used. AWS Fargate on-demand, Linux/x86: $0.000011244 per vCPU-second ($0.0405/vCPU-hour) and $0.000001235 per GB-second ($0.00445/GB-hour), read from the AWS Fargate pricing page. The page's examples are for US East (N. Virginia); I could not retrieve the Canada (Central) rate, which is higher, so the figures below are a lower bound to confirm in Cost Explorer. The shared load balancer is an assumption (about $0.0225/hour plus capacity units). OpenAI gpt-4o-mini: $0.15 per 1M input and $0.60 per 1M output tokens (OpenAI model page, which lists the model as deprecated, so treat it as a reference price).

Component	Config	Per hour	Per 30 days (730 h)
API task	0.5 vCPU, 1 GB	$0.0247	$18.02
UI task	0.25 vCPU, 0.5 GB	$0.0123	$9.01
Load balancer (assumed)	shared ALB	about $0.0225	about $16.4 + usage
ECR, logs	small		under $1 (assumed)
Total, always on		about $0.06	**about $45–46 per month, about $1.5 per day**

*Table 4. Monthly running cost estimate. **Total cloud spend to date:** $5.09 for October 2026 so far (AWS Cost and usage panel, 26 Oct 2026: Elastic Container Service, VPC, load balancing, ECR, CloudShell; the panel says credits cover free-plan costs, so this may be before credits). The project target in D1 was under $25 in total, so tearing the stack down after grading matters.*

Cost per 1,000	Basis	Value
Predictions, fixed stack cost spread over traffic	$1.5/day ÷ volume: 1,000/day	$1.50
	10,000/day	$0.15
	100,000/day	$0.015
Predictions, marginal compute only (saturated task)	35 ms of one vCPU at $0.0405/vCPU-hour	about $0.0004

Cost per 1,000	Basis	Value
Assistant queries, rule provider	retrieval in process; no vendor fee	about $0 (included in the task)
Assistant queries, OpenAI gpt-4o-mini	estimated about 1,100 input + 150 output tokens per query (system prompt and tool definitions about 430 tokens, two model calls, retrieved passages)	about $0.26 (estimate from prompt sizes, not billed usage)
Assistant queries, OpenAI embeddings for retrieval	about 20 tokens per query at $0.02 per 1M tokens (price from memory; verify)	under $0.001
Assistant queries, Ollama (local)	no per-token fee; needs a host that is on while serving	$0 marginal; fixed host cost instead

Table 5. Unit costs. The assistant token counts are estimates from the size of the prompts in the code; the deployed service has no LLM key, so no real usage exists. The quality and cost of the OpenAI and Ollama providers are not measured end to end (no key or local server in CI).

Levers. Always-on cost dominates, so the largest lever is stopping the stack when it is not being used (about $1.5/day saved). Next: one smaller task for the UI; Fargate Spot or a Savings Plan for steady use; scale-to-zero is not available for this always-warm design; for assistant spend use the cheaper model, cache frequent questions and cap output tokens. Per-prediction cost is already negligible; traffic would have to exceed hundreds of thousands per day before compute scales the bill.

5. Interpretability

Global view. `scripts/importance.py` shuffles each predictor, and each group of near-duplicate predictors, on the held-out slice and measures the drop in PR-AUC (15 repeats). Order frequency is by far the most important group (PR-AUC falls by 0.044 when shuffled), followed by spend and volume (0.008), tenure (0.005) and seasonality (0.004). Basket variety, price level and country are indistinguishable from zero. Single features understate importance because the model has many correlated copies (for example spend, revenue and quantity per order), which is why groups are used.

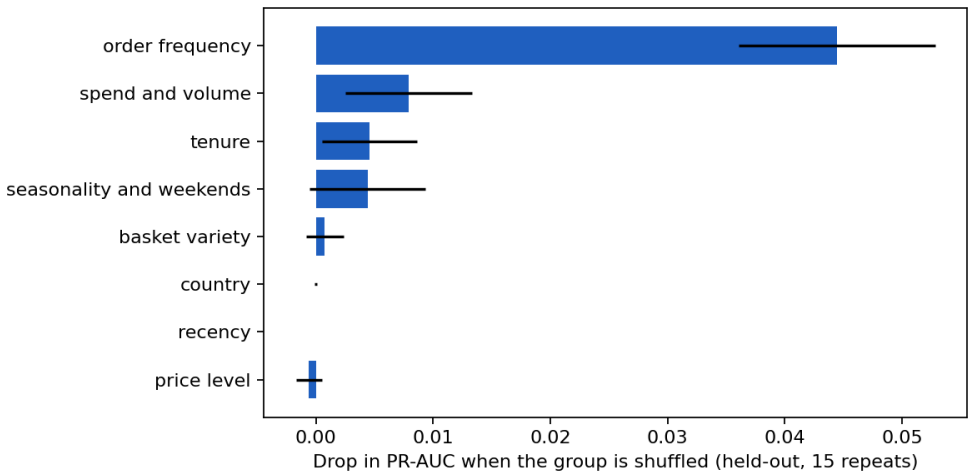


Figure 2. Global importance by feature group (held-out PR-AUC drop, mean ± sd over 15 shuffles).

Per prediction. The new endpoint `POST /explain` returns the top factor groups behind one score. Method: each group of related predictors is replaced by a typical-customer value (median of the training reference; United Kingdom for country), the customer is re-scored, and the change is reported; positive means the customer's values raise retention. It scores all groups in one batched call (about 39 ms). It uses the *uncalibrated* forest score, because the isotonic calibrator is a step function and made many groups tie exactly (a first version using calibrated scores reported identical effects of 0.117 for four groups). Example, golden customer 12347 (score 0.81): recency +5.7 points, spend and volume +4.3, seasonality +3.5, order frequency +2.9, then basket variety, tenure, price level and country below one point. The interface shows the top three for staff.

Is a customer-facing explanation honest? Only partly, so it is **not** shown to customers. (1) The effects describe how the model reacts to a value, not why the customer behaves that way; a message such as "you were flagged because your recency is low" would imply a cause. (2) Local and global views disagree: recency is the top local factor for the

golden customer but has zero global importance on the held-out slice, where little recency variation remains by construction of the chronological split. (3) Effects are in uncalibrated score points, not probability points, and small effects (below one point) are noise. (4) A group such as "country" must never appear as a reason. What is honest and useful: staff see the factor groups next to the score and may override; on a customer's request under s.12.1, a human can state the information used and the principal factors in plain language (for example "your order frequency and spend") with the caveat that a person reviewed the result.

6. Fairness audit by subgroup

Setup. Held-out slice, 788 customers, the deployed model, threshold 0.82. Positive class = retained. **Recall** = share of truly retained customers predicted retained; **precision** = share of predicted-retained customers who are retained; **selection rate** = share predicted retained (offer rate = 1 – selection rate). Groups: region (UK, Europe non-UK, rest of world), tenure band (0–90, 91–365, 366+ days) and spend quartile (cut-offs from the training slice). A group is **flagged** when its recall is more than 5 points below overall recall (54.8%). The dataset has no protected attributes, so this is a behavioural audit, not a legal fairness finding. Run `python scripts/fairness_audit.py`.

Table 6. Subgroup metrics at threshold 0.82.

Group	n	Retained (actual)	Selection rate	Recall	Recall 95% CI	Prec.	Gap vs overall	Flag
All customers	788	76%	46%	55%		91%		
Spend: Q1 (lowest)	46	41%	0%	0%	0–0%	-	-55 pts	FLAG
Spend: Q2	70	66%	0%	0%	0–0%	-	-55 pts	FLAG
Spend: Q3	116	62%	0%	0%	0–0%	-	-55 pts	FLAG
Spend: Q4 (highest)	556	83%	65%	71%	67–75%	91%	+16 pts	ok
Tenure: 0-90 days	125	62%	0%	0%	0–0%	-	-55 pts	FLAG
Tenure: 366+ days	492	78%	68%	78%	74–82%	90%	+23 pts	ok
Tenure: 91-365 days	171	79%	17%	21%	14–28%	97%	-34 pts	FLAG
Region: Europe (non-UK)	78	78%	40%	46%	33–58%	90%	-9 pts	FLAG
Region: Rest of world	5	20%	20%	100%	100–100%	100%	+45 pts	n<30
Region: UK	705	76%	47%	56%	51–60%	91%	+1 pts	ok

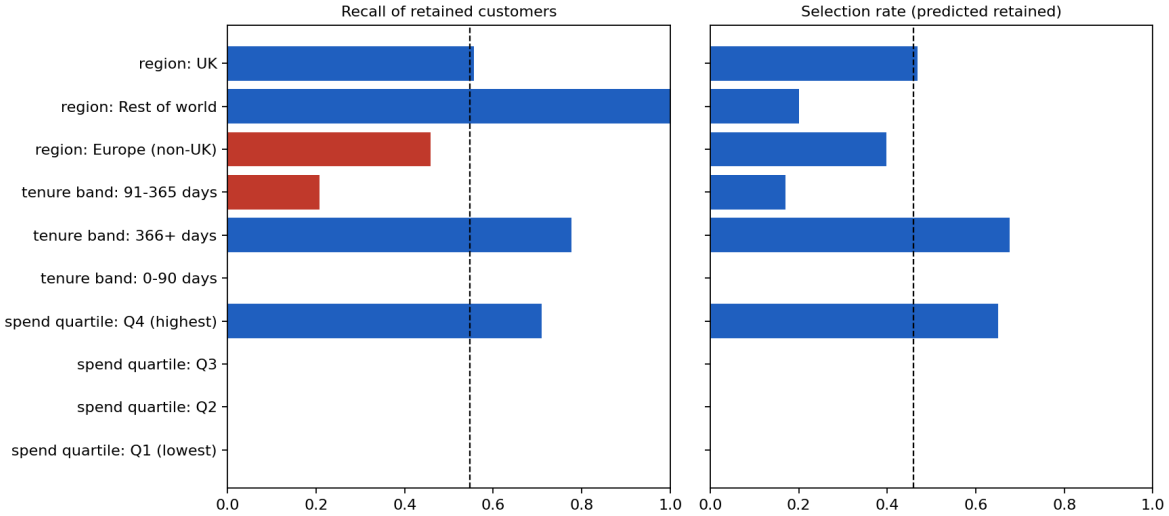


Figure 3. Recall (left, flagged groups in red) and selection rate (right). Dashed line = overall.

- **The gap is structural.** The model never scores spend quartiles Q1–Q3, new customers (0–90 days) or 1–2-order customers above 0.52, so none can pass 0.82: recall 0%, offer rate 100%. Yet 41%–66% of them were retained, so about half of their offers are unnecessary (low-spend scores are 9–13 points below the observed rate).
- **The 91–365-day band is also flagged** (recall 21%), and Europe (non-UK) is 9 points below overall but has only 78 customers and a wide interval (33–58%), so I treat it as unproven. Rest of world has 5 customers and is not

interpretable.

- **Population shift.** The held-out slice has far fewer low-spend customers (8% vs 33% in training), so group sizes are small and intervals wide. Recomputed PR-AUC on this slice is 0.8726 (metadata 0.8753; same split, cause not investigated).

Mitigations (four tested, on the same held-out slice). A is chosen on the calibration slice and applied unchanged to held-out. All figures from reports/fairness/mitigation_options.csv.

Option	Sel.	Recall	Prec.	Worst group recall	Flagged	What it does and the trade-off
Baseline, single threshold 0.82	46%	55%	91%	0%	6	Current behaviour.
A. Group thresholds by spend quartile (calibration slice)	62%	68%	83%	38%	4	Brings quartile recall to 54–70% (Q1 63%, Q2 54%, Q3 65%, Q4 70%) and cuts unnecessary offers, but precision falls to 83% (Q1 52%): about half of Q1 customers declared "retained" will churn unnoticed. Thresholds rest on small groups and shift over time: pilot only.
B. Rank within group, same review budget	46%	51%	84%	31%	4	Keeps the review workload and spreads it across groups (selection 46% in every quartile), but lowers overall recall (55% to 51%) and tenure gaps remain.
C. Per-group recalibration (isotonic)	47%	56%	91%	0%	6	**Negative result.** Almost no change: Q1–Q2 stay at 0% recall. Calibration is not the cause; the score cannot separate retained from churned customers within these groups.
D. Retrain with inverse-group weights	47%	56%	90%	0%	6	**Negative result.** PR-AUC 0.875 (no loss) but recall in Q1–Q3 stays 0%. Weighting does not add information the features lack.

Table 7. Mitigation options and trade-offs. Sel. = selection rate; Prec. = precision.

Recommendation. Keep 0.82 and mandatory human review in production (it is what stops a 100% offer rate from becoming automatic); use B as the reviewer queue ordering; pilot A for spend quartiles; and treat C and D as ruled out. The fix that would actually help is information, not tuning: features that separate retained from churned low-spend and new customers (for example the first-order basket, channel or acquisition source) need to be collected and tested. I would add a promotion check to the weekly retraining gate: no group with a 100% offer rate unless its observed retention is below 40%.

7. Law 25 governance

The full file is **GOVERNANCE_LOI25.md**; this section summarises it. **Primary sources consulted on 26 October 2026:** the text of the Act respecting the protection of personal information in the private sector (CQLR c. P-39.1) on Légis Québec, s.12.1, s.8.1 and s.23, and the Commission d'accès à l'information page on informing people about automated decisions. Section 12.1 requires that a person be informed, no later than when the decision is communicated, that a decision based exclusively on automated processing was made; that, on request, they be told which personal information was used, the reasons and the main factors and parameters, and of their right to have it corrected; and that they be given the opportunity to present observations to a staff member able to review the decision. This is not legal advice.

Decision point	Decided by	Personal information	Human review	Customer rights
Retention-risk score	Model	28 behaviour features, country	Staff see score and factor groups	Notice, explanation on request, correction
Retention offer (score < 0.82)	**Staff**, informed by the score	Same	**Mandatory approval before any offer**	Observations to the reviewing staff member
Product recommendations	Model rules	Items in the basket only	Not required (low impact)	Ignore; nothing stored
Assistant answers	Assistant	The question; policy corpus	Hand-off when no evidence	Contact staff

Table 8. Automated-decision register (short form).

- **Design consequence.** Because s.12.1 obligations attach to decisions made *exclusively* by automated processing, a human approval step is inserted before any offer, and this is the reason it exists. The API says `target_with_offer`, the interface says "a staff member reviews it", and the assistant's policy text promises review. A rubber stamp would not meet the intent: the reviewer sees the factor groups and can override. The score is also treated as profiling (s.8.1) and disclosed.
- **Customer notice in French and English** is in the About page of the application (and the repository file); wording in GOVERNANCE_LOI25.md.
- **Data-minimisation audit.** Every feature group was tested with permutation importance. Retraining without `country` and `is_uk` gives PR-AUC 0.8732 vs 0.8726, recall 55.9% vs 54.8%: no loss. **Decision: remove them at the next retraining.** Price level and basket variety are marginal and are candidates. Recency scores zero on this slice but is kept for business reasons and re-tested on a rolling window.
- **Retention and deletion.** Prediction rows (feature values, score) are kept 120 days (90-day label delay plus 30 days) and then deleted; implemented as `LogStore.purge_older_than`, run at every start-up (setting LOG_RETENTION_DAYS), with a unit test. Labels carry no features and are kept. The live prototype's log is container-local and is also wiped on every redeploy. Open: scheduled purge, erasure by request id, privacy-officer contact, consent capture.

8. Model card and data card

Both are complete files in the repository root (**MODEL_CARD.md**, **DATA_CARD.md**). The model card states intended and out-of-scope uses, training data, held-out performance, the subgroup results above, known risks (isotonic step calibration, pickle loading, library pinning) and the maintenance rule. The data card states the source and licence (to be verified against the UCI page before redistribution), cleaning stages (1,067,371 raw rows to 1,007,913 cleaned rows), composition (5,249 customers, 41 countries, 91% UK, label balance by slice), personal-information position and known biases, including that no demographic attributes exist.

9. Monitoring policy

Signal	Amber	Red	Action
Input drift: PSI per feature, with a sampling-noise correction	≥ 0.10	≥ 0.25 on any feature (raises an alert)	Alert stored (and sent to a webhook if configured), 6-hour cooldown per feature; staff check the data source first. The KS test is reported for context and does not set the status
Prediction drift: PSI on the score	≥ 0.10	≥ 0.25 (raises an alert)	Compare with input drift to separate data change from model change
Data quality per feature	out-of-range share > 5% or null rate up > 10 pts	> 20% or > 20 pts	Shown on the page and in the overall status; no alert is raised yet (proposed)
Error rate (HTTP 5xx share)	–	≥ 5% (raises an alert)	Check the last deploy; redeploy the previous image if it follows a release
p95 latency	–	≥ 500 ms (raises an alert)	Check dashboard-refresh load and task CPU
Minimum traffic for drift	–	fewer than 30 predictions: "insufficient data"	No alert; small samples can show amber from noise alone (seen at n of about 320)

Table 9. Thresholds and what they trigger (implemented in app/monitoring.py; thresholds in app/config.py).

Delayed ground truth. Retention is only observable 90 days after a prediction. Each prediction row stores its request id; true labels are backfilled with `POST /monitoring/outcomes` and joined to the prediction. The monitoring page states the date of the first true accuracy signal (about 24 January 2027 for the first live predictions) and the number of labelled predictions so far. Until then, drift and operations metrics are the only live signals, and this limitation is shown on the page rather than hidden.

Retraining decision rule. (1) *Scheduled:* a weekly workflow retrains the frozen recipe and scores candidate and deployed model on the same held-out slice. It **promotes only if** PR-AUC improves by at least 0.005, ROC-AUC drops by no more than 0.01, Brier worsens by no more than 0.005, the schema is unchanged and the data has not shrunk by

more than 10%. A promotion opens a pull request; a person merges, CI runs, and only then is it deployed. First run: keep current model (0.8782 vs 0.8764). (2) *Proposed, not yet automated:* an out-of-cycle review when red input drift persists three consecutive days, or when the first labels show PR-AUC more than 0.05 below the held-out value. (3) *Fairness gate, proposed:* add the subgroup checks of Section 6 to the promotion criteria.

10. Limitations and next steps

- Live latency was measured sequentially from one laptop and includes network time; Fargate throughput under concurrent load was not measured. AWS spend to date was $5.09 for October 2026 (Cost Explorer, 26 Oct; credits may apply); the annual cost is a projection. The in-process numbers come from a 2-vCPU development machine and the container is smaller.
- Assistant costs use estimated token counts and list prices; no LLM key is configured, so real quality and cost are not measured.
- The fairness audit uses one chronological hold-out with small groups and no protected attributes; conclusions on Europe (non-UK) and rest of world are not supported.
- Group occlusion is an approximation, not a causal explanation; it must not be shown to customers as reasons.
- Next: load-test the deployed service; remove country at retraining; add per-segment offer rates to the monitoring page; move the log to PostgreSQL; collect features that separate retained low-spend customers; add scheduled purge and erasure by request id.